

How one transaction becomes a high-risk flag

Notes on `src/fraudvae`

Recorded notebook, seed 42

Abstract

The network in this repository is trained only on synthetic normal transactions. At score time it rebuilds a row and compares that reconstruction with a cutoff taken from other normal rows. This note follows one transaction through that path, then derives the training objective the code is approximating. The rows are generated here. They are not taken from a real card-fraud dataset.

1 The question, and the table it is asked on

`ReconstructionScorer.score` answers a narrow question: is this row harder to rebuild than the noisiest 5% of a held-out normal sample? The return value is a reconstruction score and a boolean, `high_risk`. Nothing in the return value is a probability of fraud.

`generate_synthetic_transactions` draws the table. Normal rows are tied together on purpose: fee tracks amount, 24-hour rolling spend falls as the gap since the last transaction grows, device type follows distance from home, and merchant id follows spend. Fraud rows are a minority (400 of 8,000, 5%) and are shifted off those links by several standard deviations. Recall of 1.0 on the notebook means the score separated that planted gap. It is a check that the pipeline works under the generator's assumptions.

The notebook split, all disjoint by `row_id`:

Split	Normal	Fraud	Used for
Train	5200	0	weights and the scaler
Calibration	1200	0	the 95th-percentile cutoff
Validation	1200	400	the classification report

`VAETrainer` receives the label tensor and does not pass it to `TabularVAE` or `VAELoss`. The label is read again only in `detection_report`, after scores exist.

2 A row, before the network sees it

Two dicts in the notebook were passed to `score` after training. They are hand-written checks, not rows sampled by the generator. The first is ordinary relative to the generator. The second breaks the same links the fraud sampler breaks.

Field	Ordinary check	Shifted check
amount	24.5	280
fee	0.50	1.2
rolling_spend_24h	57	900
distance_from_home_km	3	80
seconds_since_last_txn	6600	80000
merchant_id	m3	m8
device_type	contactless	chip

For scale, the notebook also printed medians of the *generated* rows. Those are outputs of the sampler, not of these two dicts.

Generated median	Normal	Fraud
amount	24.47	267.91
fee	0.50	1.33
rolling_spend_24h	57.11	898.06
distance_from_home_km	3.05	79.67
seconds_since_last_txn	6708	71842

The ordinary check sits next to the normal medians. The shifted check sits next to the fraud medians. That is why they are useful probes. Section 8 records what `score` actually returned for them.

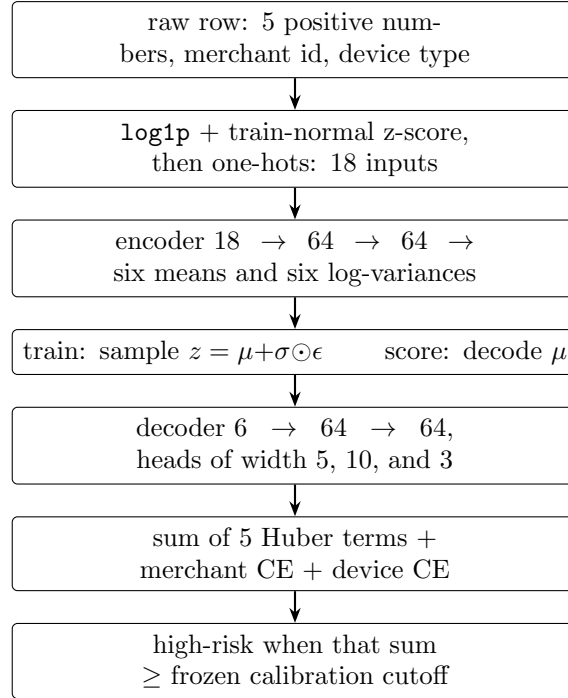


Figure 1: Path of one row. The cutoff is a stored number from the calibration split, not a network output.

2.1 Symbols used below

Symbol	Meaning here
u	The five raw continuous fields, in the column order above. Each entry is > 0 .
c_m, c_d	Class indices. <code>m0</code> is 0. Device order is <code>chip</code> , <code>contactless</code> , <code>online</code> .
$\tilde{u}$	u after <code>log1p</code> and the training-normal z-score.
$x \in \mathbb{R}^{18}$	$\tilde{u}$, then the merchant one-hot, then the device one-hot.
$\mu, \logvar \in \mathbb{R}^6$	Encoder outputs. They parameterize a Gaussian, they are not the code itself.
$z \in \mathbb{R}^6$	A latent code. Training samples it. Scoring uses μ .
$q_\phi(z x)$	Encoder distribution. ϕ is the encoder weights.
$p(z)$	$\mathcal{N}(0, I_6)$. Fixed.
$p_\theta(x z)$	Decoder distribution in the derivation. The loss replaces it with Huber and cross-entropy.

3 Eighteen numbers

`ContinuousPreprocessor.fit` sees only the 5,200 training normals. For each continuous column it stores the mean and the population standard deviation of $\log(1 + u)$. Later rows, including calibration, validation, and anything passed to `score`, reuse those two vectors. A non-finite or non-positive value is rejected. Zero is not clamped to a small epsilon.

The notebook printed the fitted vectors rounded to three decimals:

$$\begin{aligned} m &= (3.246, 0.432, 4.068, 1.420, 8.813), \\ s &= (0.478, 0.179, 0.563, 0.375, 0.504). \end{aligned}$$

Applying that printed scaler to the ordinary check is an illustration, not a dump of the tensor the model stored:

$$\tilde{u}_{\text{ordinary}} \approx (-0.02, -0.15, -0.01, -0.09, -0.04).$$

The same arithmetic on the shifted check gives roughly (5.0, 2.0, 4.9, 7.9, 4.9). The recorded validation means, which *are* model inputs averaged over generated rows, agree with the picture: normals sat near $(-0.04, -0.04, -0.03, 0.00, -0.01)$ and fraud near (4.94, 2.33, 4.84, 7.94, 4.70).

Categories are not scaled.

`encode_categoricals` maps each token to an index.

`TabularVAE.encode` one-hots that index. `m3` is index 3 of ten. `contactless` is index 1. Device order is `chip`, then `contactless`, then `online`. The encoder input is the concatenation

$$x = [\tilde{u} \mid e_3^{(10)} \mid e_1^{(3)}] \in \mathbb{R}^{18}$$

for the ordinary check, and the analogous vector with $e_8^{(10)}$ and $e_0^{(3)}$ for the shifted check. The 18 is $5 + 10 + 3$. Swapping vocab order would train a different model than the one `score` applies. An unknown token raises. It does not become a zero one-hot, and it is not automatically high-risk.

4 Six means and six log-variances

The encoder trunk is two affine maps with a ReLU after each:

$$\begin{aligned} h_1 &= \text{ReLU}(W_1 x + b_1), \quad W_1 \in \mathbb{R}^{64 \times 18}, \\ h_2 &= \text{ReLU}(W_2 h_1 + b_2), \quad W_2 \in \mathbb{R}^{64 \times 64}. \end{aligned}$$

Two linear maps with no activation read h_2 :

$$\mu = W_\mu h_2 + b_\mu, \quad \text{logvar} = \text{clamp}(W_\lambda h_2 + b_\lambda, -10, 10).$$

Both results have length 6. The clamp only stops $\exp(\text{logvar})$ from overflowing. It is not a model of the data.

Those twelve numbers describe a region, not a point. The approximate posterior is diagonal:

$$q_\phi(z | x) = \mathcal{N}(\mu, \text{diag}(\exp(\text{logvar}))).$$

μ is the center. $\exp(\text{logvar}_j)$ is the variance along coordinate j . A confident encoder puts a tight blob on a normal row. A row unlike the training normals can produce a center far from the origin, a wide blob, or both. The network is not asked to emit a single “correct” z .

5 Why training samples, and scoring does not

The reconstruction loss has to be computed at a code drawn from $q_\phi(z | x)$, and the draw has to carry a gradient back into μ and logvar . Sampling z inside the Gaussian and then differentiating through the sample does not work: the draw itself is not a function the autograd graph can see. The reparameterization writes the same draw as a deterministic function of noise that does not depend on ϕ :

$$\sigma = \exp(\tfrac{1}{2} \text{logvar}), \quad \epsilon \sim \mathcal{N}(0, I_6), \quad z = \mu + \sigma \odot \epsilon.$$

`TabularVAE.forward` is that path: encode, sample, decode. The KL term never needs ϵ . It is a closed function of μ and logvar . The sample exists so a bad reconstruction can move the Gaussian that produced the code.

`ReconstructionScorer` never calls `reparameterize`. It decodes μ . The ordinary check then scores the same way on every call. Drawing a fresh ϵ would move the score even when the weights and the row were fixed, and the cutoff stored from calibration would not refer to the same quantity.

6 Rebuilding numbers and rebuilding labels

The decoder maps $z \in \mathbb{R}^6$ through the same kind of trunk, in the other direction:

$$\begin{aligned} g_1 &= \text{ReLU}(W_3 z + b_3), & W_3 &\in \mathbb{R}^{64 \times 6}, \\ g_2 &= \text{ReLU}(W_4 g_1 + b_4), & W_4 &\in \mathbb{R}^{64 \times 64}. \end{aligned}$$

Three linear heads read g_2 . None applies a softmax or a sigmoid.

Head	Shape	What it is asked to match
<code>continuous_head</code>	$64 \rightarrow 5$	$\tilde{u}$: scaled amount, fee, rolling spend, distance, seconds
merchant logits	$64 \rightarrow 10$	the merchant index, via cross-entropy
device logits	$64 \rightarrow 3$	the device index, via cross-entropy

The split is there because the two kinds of fields are not the same mistake. Huber compares two real numbers in the scaled space. With $\delta = 1$, a residual inside one training standard deviation is quadratic, and a larger residual has constant slope:

$$\ell_1(r) = \begin{cases} \frac{1}{2} r^2 & \text{if } |r| \leq 1, \\ |r| - \frac{1}{2} & \text{if } |r| > 1. \end{cases}$$

Squared error on a raw amount would let a few large but legitimate tickets dominate the gradient. `log1p` and the z-score shrink that problem. Huber is what remains so a tail residual still cannot grow like r^2 .

Cross-entropy compares a distribution over class indices with the index that was observed. `F.cross_entropy` applies the log-softmax to the raw logits. A squared error on the merchant index would treat `m9` as nine times `m1`. The vocabulary has no such order. `m3` versus `m8` is an identity error, which is what the merchant head is for. The same holds for `chip` versus `contactless`.

On a normal row the continuous head is mostly checking the links in $\tilde{u}$ (fee against amount, rolling spend against the gap). The categorical heads are checking whether the merchant and device are the ones that usually accompany that pattern. A shifted row can fail either check or both.

7 Why only normal rows are shown to the loss

The weights never see a fraud row, so the decoder’s job is only to rebuild normal traffic inside the latent region the KL allows. After training, every row still has an encoder output and a decoder output. For a normal validation row, both were trained on rows like it, and the residual is usually small. For a generated fraud row, the decoder’s best reconstruction is still a normal-looking combination: a fee that matches the amount, a device that matches the distance, a merchant that matches the spend. The inputs disagree with that combination, so Huber and cross-entropy rise.

That is the whole detection idea. It fails in two directions that matter later. A fraudulent row that happens to look normal in these seven fields reconstructs easily and stays under the cutoff. A legitimate row that is merely unfamiliar (a new distance, a rare but honest merchant) can land above it. The score is conformity to the learned normal structure. It is not evidence of intent.

If fraud were left in the training normals, the decoder would spend capacity on those patterns and the gap would shrink. The notebook split checks that train and calibration are entirely label 0. That check is not available once the label is something you do not have.

8 One number per row, then a frozen cutoff

Training and scoring use the same Huber delta and the same λ_{cont} and λ_{cat} . They do not use the same reduction.

`VAELoss` averages so a change in batch size does not rescale the gradient. Huber is `F.huber_loss` with `reduction="mean"`, which averages over the batch and over the five columns. Each head’s cross-entropy is averaged over the batch, then the two head-means are added. The KL is summed over the six latent coordinates and then averaged over the batch.

`reconstruction_errors` sums inside the row instead. A transaction needs one severity, not a batch mean:

$$\begin{aligned}\text{Huber}_{\text{sum}} &= \sum_{j=1}^5 \ell_1(\hat{u}_j - \tilde{u}_j), \\ \text{CE}_{\text{sum}} &= \text{CE}(\text{logits}_{\text{m}}, c_{\text{m}}) + \text{CE}(\text{logits}_{\text{d}}, c_{\text{d}}), \\ \text{score} &= 1 \cdot \text{Huber}_{\text{sum}} + 1 \cdot \text{CE}_{\text{sum}}.\end{aligned}$$

KL is not in this sum. The decode is of μ .

`fit.threshold` runs that score on the 1,200 calibration normals and stores the 95th percentile. The notebook printed 2.1162 (the float kept on the scorer was 2.116162061691284). `high_risk` is `score ≥ threshold`. Later calls read the stored float. They do not take a new percentile of

the request, and they do not take one of the validation set. The calibration rows are not in the classification report.

On the recorded run the score distributions were:

Slice	n	mean	p50	p95	max
Calibration normals	1200	0.843	0.694	2.116	4.259
Validation normals	1200	0.818	0.667	2.040	4.037
Validation fraud	400	23.257	24.344	28.517	30.878

Fraud scores started at 13.189. All 400 sat above 2.1162, so fraud recall is 1.0 and fraud precision is 0.883. The precision is below 1 because some validation normals also crossed the line: the notebook’s flag rate on them was 0.044. That is 4.4%, close to 5% and not equal to it. The cutoff is an order statistic of one sample of 1,200 normals. The validation normals are a different sample. Their own 95th percentile was 2.040, below the calibration cutoff, so fewer than one in twenty of them were flagged. The two rates would match only if the two empirical percentiles were the same number.

The two hand-written checks, which are actual `score` outputs: the ordinary row scored 0.052 (continuous error 0.051, both categorical errors near 0) and was not high-risk. The shifted row scored 25.95 (continuous 15.90, merchant 3.79, device 6.26) and was high-risk. Those categorical pieces are cross-entropies of the true class, not accuracies. A device term of 6.26 means the `chip` logit was far below the alternatives, which is what a far-from-home chip purchase looks like to a decoder trained on local chip use.

9 The objective, after the path is clear

An autoencoder only minimizes reconstruction. With enough capacity it can give a strange row its own latent island and rebuild it perfectly. The island does not have to lie near the codes of ordinary rows, and a point halfway between two islands does not have to decode to anything normal-looking. A VAE adds a cost for that. The encoder must place $q_\phi(z | x)$ near $\mathcal{N}(0, I_6)$, so the codes of normal rows share one region. Interpolation inside that region is then a mix of normal reconstructions rather than a jump between memorized points.

That is the reason this repo uses the KL term. It is not evidence that a VAE will beat an autoencoder on every anomaly problem. If the bottleneck is already tight, or if the anomalies are obvious in the input, the KL may change little. If β is large, reconstruction of the normals themselves gets worse and the cutoff becomes a blur.

The marginal likelihood makes the same point in one integral. A decoder $p_\theta(x | z)$ and a prior $p(z) = \mathcal{N}(0, I_6)$ define

$$\log p_\theta(x) = \log \int p_\theta(x | z) p(z) dz.$$

The integral is over $\mathbb{R}^6$, and the decoder is a stack of ReLUs, so there is no closed form to differentiate. Insert the encoder density, which integrates to one, and apply Jensen’s inequality to the logarithm:

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi}[\log p_\theta(x | z)] - \text{KL}(q_\phi(z | x) \| p(z)). \quad (1)$$

The right-hand side is the ELBO. The expectation is the reconstruction term: the code just drawn has to explain the row. The KL is the pressure that keeps those codes in one region.

`VAELoss` does not implement the negative of (1) with a single density. There is no learned observation variance, and Huber is not a Gaussian log-likelihood. Cross-entropy on a head *is* the

negative log-likelihood of a categorical distribution over that head’s classes. The KL *is* the exact diagonal-Gaussian formula, averaged over the batch:

$$\text{KL}_i = -\frac{1}{2} \sum_{j=1}^6 \left(1 + \log \text{var}_{i,j} - \mu_{i,j}^2 - \exp(\log \text{var}_{i,j}) \right), \quad \text{KL} = \frac{1}{B} \sum_{i=1}^B \text{KL}_i.$$

The quantity that is differentiated is a weighted surrogate:

$$\text{total} = \lambda_{\text{cont}} \text{Huber} + \lambda_{\text{cat}} (\text{CE}_m + \text{CE}_d) + \beta \text{KL},$$

with $\lambda_{\text{cont}} = \lambda_{\text{cat}} = 1$ and $\beta = 0.1$. Huber plus the two cross-entropies stand in for $-\mathbb{E}[\log p_\theta(x | z)]$. They are not one joint density evaluated at x .

$\beta = 0.1$ is the trade. The ELBO uses weight 1 on the KL. That weight is large enough, early in training, to collapse q_ϕ toward the prior before the decoder has learned the normal links. A smaller β lets reconstruction move first. It does not keep the KL small forever. At epoch 25 the printed pieces were Huber 0.2116, cross-entropy 0.0861, and KL 5.8136, so the weighted terms were 0.2116, 0.0861, and $\beta \cdot \text{KL} = 0.5814$. The regularizer was the largest term in the training loss. The score in Section 8 still omits it.

Leaving KL out is a choice of signal, not a claim that posterior anomaly scores are useless. One could threshold the KL itself, or the likelihood of z under the prior, or a density estimated on the normal codes. Those scores answer “how strange is the latent distribution for this row.” This repository answers “how badly did the normal-trained decoder rebuild this row,” and the calibration percentile is a percentile of that rebuild error only.

10 What the perfect recall does not mean

Generated fraud is easy for this score. The mean scaled fraud row sits several standard deviations from the normal cloud, and the minimum fraud score in validation was 13.2 against a cutoff of 2.12. A bug that dropped the fraud rows into the normal cloud would have failed this test. Passing it does not measure performance on real payments, because no real payments were used.

Other limits are structural:

- The category sets are closed. A new merchant id is rejected in **score**. It is not given a score and it is not labeled fraud.
- Each row is independent. There is no cardholder, no account, and no sequence. The gap and the rolling spend are columns, not a hidden state carried across time.
- The cutoff drifts when normal behavior drifts. 2.1162 belongs to this calibration window. A later window needs its own percentile, fit on normals that were not used to train, and then frozen again.
- The score is not a probability. Crossing 2.1162 means “at or above the stored percentile,” which on this run flagged 4.4% of a second normal sample and 100% of the synthetic fraud.
- Normal training data that secretly contains fraud teaches the decoder to rebuild those fraud patterns. The synthetic split forbids that. A production extract has to enforce it some other way.

11 Where this is in the code

The notebook calls these in order. The shape check before `fit` decodes μ and does not draw ϵ , so it does not use up the noise the trainer is about to sample.

Call	What it does on this row
<code>dataset.py</code>	Builds the table, fits the scaler on train normals, and yields scaled columns plus class indices. The training loader's labels are all 0.
<code>TabularVAE.forward</code>	Concatenates to 18, writes $(\mu, \logvar)$, samples z , and decodes the three heads.
<code>VAELoss</code>	Mean Huber, summed per-head mean cross-entropy, batch-mean KL, then <code>total</code> with the three weights.
<code>VAETrainer.fit</code>	Adam for 25 epochs. The label is unpacked as <code>_labels</code> and unused.
<code>ReconstructionScorer</code>	<code>fit_threshold</code> stores 2.1162. <code>score</code> preprocesses, decodes μ , sums the row, and compares.